

## LESSON GUIDE

# Technology Plus

## Unit 2: Programming

### 4.3.3 – Functions

#### Lesson Overview:

In this lesson, students will gain firsthand experience with functions and understand how they improve the program development process.

#### Students will:

- Discuss the benefits of writing functions.
- Analyze the development and use of functions within a larger programming project.

**Guiding Question:** How do functions support efficient programming practices?

**Suggested Grade Levels:** 8-12

**Technology Needed:** Yes

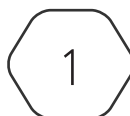
**Approximate Time Required:** 40-65 minutes

#### Standards:

##### CompTIA Tech+ FC0-U71 Objective

- 4.3 – Explain the purpose and use of programming concepts.
  - Identifiers
    - Variables
    - Constants
  - Arrays
  - **Functions**
  - Objects
    - Properties
    - Attributes
    - Methods

*This content is based upon work supported by the US Department of Homeland Security's Cybersecurity & Infrastructure Security Agency under the Cybersecurity Education Training and Assistance Program (CETAP).*



**CYBER.ORG**  
THE ACADEMIC INITIATIVE OF THE CYBER INNOVATION CENTER

Copyright © 2025 Cyber Innovation Center  
All Rights Reserved. Not for Distribution.

# Lesson Background

---

## Background Information:

This lesson and lab will require some basic understanding of reading code, primarily in Python. Students will focus on the use of functions as a tool in program development. Remind them about functional programming as a style of programming introduced in 4.1.1 – Programming Language Categories.

## Materials Needed:

|                       |                                                                                                                                                                                 |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Materials per Class   | <ul style="list-style-type: none"><li>• Lesson Guide</li><li>• Lesson Slides 4.3.3 – Functions</li><li>• Lab Slides 4.3.3 – Fruitful Functions</li></ul>                        |
| Materials per Student | <ul style="list-style-type: none"><li>• Student Notes 4.3.3 – Functions</li><li>• Student Handout 4.3.3 – Functions</li><li>• Assessment 4.3.3 – Functions (optional)</li></ul> |

## Cyber Terms:

| Term          | Definition                                                           |
|---------------|----------------------------------------------------------------------|
| function      | a reusable piece of code that performs a specific action when called |
| function call | a request to run a function by using its name                        |

# 4.3.3 – Functions

**Guiding Question:** How do functions support efficient programming practices?

---

## Lesson Launch (5-10 minutes)

- Distribute Student Handout 4.3.3 – Functions and encourage students to fill in the answers during the lesson.
- **Ask** students to provide a definition for function.  
*Student responses will vary but will likely mention mathematical concepts such as inputs and outputs.*
- Define **function** as a reusable piece of code that performs a specific action when called.
- Discuss the examples. Ask students about the “hidden” code for each example.
- If students are unfamiliar with functions, the second example may be tricky. Consider skipping it or providing hints. Also, remind them about manipulating Strings as covered in 4.2.1 – Fundamental Data Types.

## The Function of Functions (15-25 minutes)

### What’s In a Function?

- Input: information given to the function; also called parameters or arguments.
- Processing: code inside the function that changes or uses the input.
- Output: a result of the function.
- Not all functions take explicit input or produce an explicit return value. Inputs might be default to the function. Outputs might be background actions.

### Why Do We Use Functions

- Simplify Code
  - Functions break larger problems into smaller pieces.
  - Explain the example. Instead of writing the Pythagorean function as `math.sqrt(a*a + b*b)`, using a function called square helps separate the calculations being performed.
  - Benefit: Less repetition in typed code; fewer mistakes.
- Makes it Easier to Reuse Code
  - Avoid copying and pasting identical blocks of code.
  - Write it once, then name it, save it, and use it in many scenarios.
  - Explain the example. First code block retypes the greeting each time. Second code block defines the `greet()` function so it is reusable, and the outputs can be stored easily for reuse later.
  - Benefit: Generalizes a problem so one function works with any input.

- Improves Readability
  - Descriptive names communicate what is happening in the code.
  - Benefit: Others using your program, or even if you need it again in the future, have an easier time reading and understanding it.
- Streamlines the Editing and Revising Process
  - Allows changes to be made in one place and applied in many places.
  - Supports debugging by testing each function separately before combining into one larger program.
  - Animations show fixing errors in the first code block in two places when square is not defined versus the second code block only fixing it once within the square function definition.
  - Benefit: Isolating review to individual code segments allows for faster review, testing, and editing.

## Setting Up Functions

- The example referenced for Function Setup is a program called **passwordStrength()** that checks the strength of a password based on four requirements (length, uppercase letters, lowercase letters, and numbers/special characters). While only code snippets are shown for discussion, the full program can be accessed in the lab folder for this lesson in the CYBER.ORG Range.
- Keywords – different languages have syntax and reserved words for function definition.
  - Python uses **def**.
  - Java needs access modifiers (public, private, or protected), the static keyword to indicate it can be called without an object reference, and a return type (data type or void).
- Name – like variables, function names should be descriptive of what the function does.
  - **Ask** students to predict the purpose of a function called **passwordStrength()**.
- Inputs/parameters/arguments – information the function needs to complete its task.
  - A helper function, **strengthRating()**, is created to take in a number from the given password and test it against a set requirement.
- Body – where the processes are written for the function.
  - The body of **strengthRating()** shows how it tests the number against the requirement with less than, equal to, or else.
- Results – outputs, return values, or behaviors of the function completing its task.
  - **strengthRating()** returns a rating of “weak”, “medium”, or “strong” based on the conditions.
- **Call** – invoking the function using its name and parentheses, making sure to include any inputs in the parentheses.
  - **strengthRating()** is called within another function, **lenStrength()**, to compare the length of a password to a requirement of 14 characters: **strengthRating(length, 14)**

## Lab: 4.3.3 Fruitful Functions (15-25 minutes)

- This lab requires access to the CYBER.ORG Range, the Windows Server 22 Machine, and Lab Slides 4.3.3 – Fruitful Functions.
- The lab can be performed as a class activity or an independent student assignment.
- Students should record responses to reflection prompts in their Student Handout.
- Emphasize the importance of careful typing to reduce errors.
- If you and/or students get a message about the calculator.py file opening in Restricted Mode, click Manage then Trust. **The program will not run in Restricted Mode.**
- Students will be responsible for filling in code for four helper functions (add, subtract, multiply, and divide). The divide function will require an if-statement to avoid errors related to division by zero.
- Slides 9 and 10 offer two possible extension opportunities. Feel free to use as appropriate.

### Putting It All Together

- Why did we write the **add**, **subtract**, **multiply**, and **divide** functions instead of just using **num1 + num2** inside the **calc** function?

The functions allow us to avoid repeating code, to update the program easily, and to improve readability. We can also store returned values for later use.

- Why do you think it is important to check for division by zero inside the **divide** function itself?

Checking for division by zero inside the function organizes the logic and keeps error handling consistent for all division problems.

- How does the **calc\_add\_on** code demonstrate benefits of functions?

**calc\_add\_on** implements the **calc** function, which implements each math operation, into one concise program for a user to quickly and repeatedly perform basic calculations.

### Putting It All Together (5 minutes)

- **Ask:** What are the main purposes of functions?

Simplify code, reuse code, improve readability, and modify and debug code.

- **Ask:** Why do functions use parentheses?

Parentheses show that it is a function and indicate where to include inputs, if there are any.

- **Ask:** How do you invoke the following function with input “tomato”? What does it return?

```
def duplicate(word):  
    return word + word
```

**print(duplicate("tomato"))** → “tomatotomato”.